Chapter 4: CSS (Cascading Style Sheets)

1 Introduction to CSS

CSS (Cascading Style Sheets) is a W3C-standard language created in the 1990s to control webpage appearance (layout, colors, fonts, etc.). It works alongside HTML but keeps **content (HTML)** and **design (CSS)** separate.

1.1 Key Features:

- Controls layout, positioning, colors, text size.
- Works with HTML but is a distinct language.

1.2 CSS Versions

Version	Year	Key Features	Example
CSS1	1996	Basic styling (colors, margins, fonts)	p { margin: 10px; }
CSS2	1999	Advanced positioning (absolute, relative)	div { position: absolute; }
CSS3	Modern	Shadows, gradients, animations	button { border-radius: 8px; }

1.3 CSS Syntax

1.3.1 Basic Rule Structure

```
selector {
  property: value;  /* Declaration */
}
```

Example:

```
h1 {
  color: navy;
  font-family: Arial;
}
```

1.4 Adding CSS to HTML

1.4.1 Inline

Styles applied directly to an HTML element using the style attribute.

```
<p style="color: blue; font-size: 16px;">This is a paragraph.</p>
```

✕ **Disadvantage:** Mixes content and styling.

1.4.2 Internal

Styles defined within a <style> tag in the <head> section.

```
<head>
  <style>
    p {
      color: blue;
      font-size: 16px;
    }
  </style>
</head>
```

✓ **Advantage:** Good for single-page styling.
✕ **Disadvantage:** Only applies to one page.

1.4.3 External

Styles written in a separate .css file and linked to the HTML document.

```
<head>
  <link rel="stylesheet" href="styles.css">
</head>
```

✓ **Best Practice:** Reusable across multiple pages.

2 Types of Selectors

2.1 Tag Selector (e.g., p, h1)

```
p { color: blue; }
```

2.2 Class Selector (.class)

```
.highlight { background: yellow; }
```

2.3 ID Selector (#id)

```
#header { font-size: 24px; }
```

2.4 Universal Selector (* : all elements)

```
* { background-color: yellow; }/* background color of all elements */
```

Possibility to modify this background color for a particular element:

```
/* Special case */
.p { background-color: white; }
```

2.5 Select and Style Multiple Elements

To apply the **same style to different elements**, list them with **commas**:

```
selector1, selector2, selector3 {
  property: value;
}
```

Example:

```
/* Applies to ALL h1, h2, and h3 elements */
h1, h2, h3 {
  color: navy;
  background-color: gray;
}
```

This is equivalent to:

```
h1 { color: navy; background-color: gray; }
h2 { color: navy; background-color: gray; }
h3 { color: navy; background-color: gray; }
```

2.6 Advanced Selectors

2.6.1 Contextual Selectors

*2.6.1.1 **Parent-Descendant** (parent descendant)*

```
li li { font-size: 11px; }  /* Nested <li> only */
```

*2.6.1.2 Direct Child **(parent > child)***

```
p > img { float: right; }  /* Only images inside <p> */
```

*2.6.1.3 Adjacent Sibling **(element + sibling)***

```
h1 + p { font-style: italic; }  /* Paragraph after <h1> */
```

2.6.2 Pseudo-Classes

```
a:hover { color: orange; }  /* Hover effect */
```

2.6.2.1 Link-Specific Pseudo-Classes

(For elements with href attributes)

Pseudo-Class	Description	Example
:link	Styles unvisited links	a:link { color: blue; }
:visited	Styles visited links	a:visited { color: red; }

Example:

```
/* Unvisited links = blue */
a:link { color: blue; }

/* Visited links = red */
a:visited { color: red; }
```

2.6.2.2 Dynamic Pseudo-Classes

(Change style based on user interaction)

Pseudo-Class	Trigger	Example
:hover	Mouse over element	a:hover { text-decoration: underline; }
:active	Element being clicked	button:active { transform: scale(0.95); }
:focus	Element selected (e.g., via keyboard)	input:focus { border-color: green; }

Example:

```
/* Hover effect */
a:hover { background: gold; }

/* Click effect */
button:active { background: darkred; }

/* Keyboard focus */
input:focus { outline: 2px solid lime; }
```

2.6.2.3 Other Useful Pseudo-Classes

Pseudo-Class	Targets	Example
:first-child	First child of a parent	li:first-child { font-weight: bold; }
:lang()	Elements with a language attribute	p:lang(fr) { color: black; }

Example:

```
/* Style first item in a list */
ul li:first-child { color: purple; }

/* Style French paragraphs differently */
p:lang(fr) { font-style: italic; }
```

2.6.3 Key Takeaways

✅ **Links:** use :link and :visited for navigation styling.
✅ **Interactivity:** :hover, :active, and :focus improve UX.
✅ **Advanced Targeting:** :first-child and :lang() offer precise control.

Full Example:

```
/* Link states */
a:link { color: navy; }
a:visited { color: gray; }
a:hover { background: yellow; }
a:active { color: red; }

/* Form focus */
input:focus { background: lightblue; }

/* Language-specific */
p:lang(en) { font-family: Arial; }
```

2.6.4 **Pseudo-Element** (styling parts of an element)

Pseudo-elements target **specific parts of an element** (e.g., first letter, placeholder text).

```
p::first-line { color: blue; }  /* First line of <p> */
```

2.6.4.1 Key Rules:

- Use **::** (double colon) in CSS3 (though : still works for backward compatibility).
- Only **one pseudo-element per selector**.
- Must be placed **after simple selectors** (e.g., p::first-line, not ::first-line p).

2.6.4.2 Common Pseudo-Elements:

Pseudo-Element	**Targets**	**Example**	**Browser Support**
::after	Inserts content *after* an element	.box::after { content: "→"; }	All browsers
::before	Inserts content *before* an element	.note::before { content: "!"; }	All browsers
::first-letter	First letter of text	p::first-letter { font-size: 200%; }	All browsers
::first-line	First line of text	p::first-line { font-weight: bold; }	All browsers
::selection	Highlighted text	::selection { background: yellow; }	All browsers

Pseudo-Element	Targets	Example	Browser Support
::placeholder	Input placeholder text	input::placeholder { color: gray; }	Modern browsers
::marker	List item bullets/numbers	li::marker { color: red; }	Modern browsers
::backdrop	Background behind modal dialogs	dialog::backdrop { background: rgba(0,0,0,0.5); }	Modern browsers
::grammar-error	Grammar mistakes (experimental)	::grammar-error { text-decoration: red wavy underline; }	Limited
::spelling-error	Spelling mistakes (experimental)	::spelling-error { text-decoration: red wavy underline; }	Limited

Example:

```
/* Blue first line for paragraphs */
p::first-line { color: blue; }

/* Add an icon before links */
a::before { content: "🔗 "; }
```

2.6.5 The !important Rule

Overrides all other conflicting styles for a property.

Example:

```
* {
  color: black !important; /* Forces text to be black */
  background: yellow;      /* Normal declaration */
}
div {
  color: blue;             /* Ignored due to !important */
  background: white;       /* Applied (no !important conflict) */
}
```

Result:

- Text color: **black** (from * selector, due to !important).
- Background: **white** (from div selector, no !important conflict).

When to Avoid !important**:**

✕ Makes debugging harder (breaks the natural CSS cascade).
✕ Leads to "specificity wars" in large projects.

Better Alternatives:
✅ Increase specificity (e.g., use div.special instead of div).
✅ Refactor CSS structure to avoid conflicts.

2.6.6 Key Takeaways

1. **Pseudo-elements** style *parts* of elements (use ::).
2. **!important** overrides all styles but should be used sparingly.
3. **Best practice:** Prioritize clean CSS structure over !important.

Example of Good Practice:

```
/* Specific selector beats !important */
header.navbar h1 { color: navy; }

/* Instead of: */
h1 { color: red !important; } /* Avoid unless critical */
```

2.6.7 Attribute Selectors

2.6.7.1 Basic Attribute Selectors

Selector	Targets	Example	Use Case
[attr]	Elements with the attribute	a[title]	Links with a title attribute
[attr=value]	Exact attribute value	input[type="email"]	Email input fields
[attr~=value]	Attribute contains word (space-separated)	img[alt~="logo"]	Images with "logo" in alt text
[attr|=value]	Attribute starts with value (or value-)	[lang|="en"]	English content (en-US, en-GB)
[attr^=value]	Attribute starts with value	a[href^="https"]	Secure HTTPS links
[attr$=value]	Attribute ends with value	a[href$=".pdf"]	PDF file links
[attr*=value]	Attribute contains substring	[class*="btn-"]	All button variants

2.6.7.2 Practical Examples

```
/* Target download links */
a[download] { color: green; }

/* Style external links */
a[href^="http://"]::after { content: " ↗"; }

/* Highlight secure sites */
a[href^="https://"] { border-left: 3px solid green; }

/* Style PDF icons */
a[href$=".pdf"]::before { content: "📄 "; }
```

```
/* Mobile-first inputs */
input[type="tel"],
input[type="email"] {
  width: 100%;
}
```

2.6.7.3 Advanced Combinations

```
/* Links containing "example.com" */
a[href*="example.com"] {
  background: yellow;
}
/* Images with both alt AND title */
img[alt][title] {
  border: 2px solid red;
}
/* Language-specific styling */
[lang|="zh"] { font-family: "SimSun"; }  /* Chinese */
[lang="fr"] { font-style: italic; }      /* French */
```

2.6.7.4 Key Benefits

✅ **Precise targeting** without extra classes
✅ **Maintainable** HTML (no class clutter)
✅ **Powerful form styling** (e.g., input[required])

2.6.7.5 Browser Support

All modern browsers support attribute selectors, including:

- Chrome, Firefox, Safari, Edge (all versions)
- IE7+ (except some edge cases in IE7)

2.6.7.6 Pro Tips

1. **Combine with pseudo-elements**:

   ```
   a[href^="tel:"]::before { content: "📞 "; }
   ```

2. **Use for validation**:

   ```
   input:invalid { border-color: red; }
   ```

3. **Case sensitivity**:
   - [attr="value"] is case-sensitive
   - Add i flag for case-insensitive matching (CSS4):

     ```
     [class="btn" i]  /* Matches "btn", "BTN", "Btn" */
     ```

3 CSS Properties

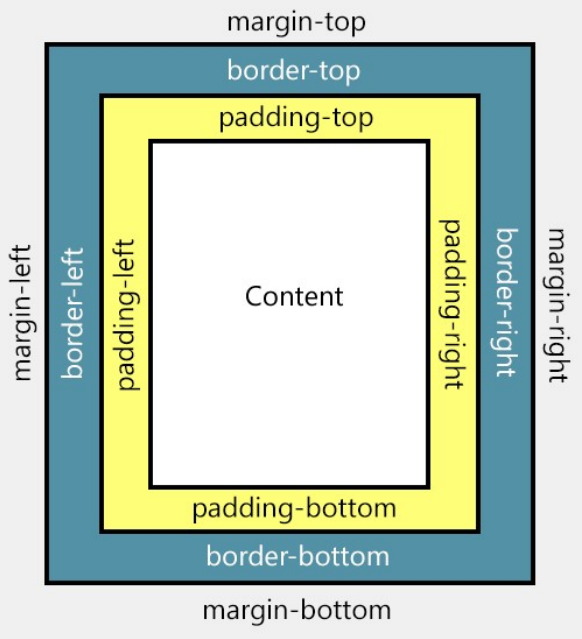
3.1 Text Properties

Property	Description	Example
font-size	Text size	font-size: 16px;
font-style	Italic/normal	font-style: italic;
font-family	Font type	font-family: Arial;
font-weight	Boldness	font-weight: bold;

3.2 Paragraph Properties

Property	Description	Example
line-height	Space between lines	line-height: 1.5;
text-align	Horizontal alignment	text-align: center;
text-indent	First-line indent	text-indent: 20px;
text-transform	Text casing	text-transform: uppercase;

3.3 Box Model Properties



Explanation of the different parts:

- **Content** - The content of the box, where text and images appear
- **Padding** - Clears an area around the content. The padding is transparent
- **Border** - A border that goes around the padding and content
- **Margin** - Clears an area outside the border. The margin is transparent

Property	Description	Example
width/height	Element dimensions	width: 100px;
margin-*	Outer spacing (Space outside the border)	margin-top: 10px;
padding-*	Inner spacing (Space between the content and the border)	padding-left: 15px;
border-*	Border styles	border-style: dotted;

* : **top**, **bottom**, **left** or **right**

4 CSS Units

4.1 Absolute Units

Unit	Description	Use Case
px	Pixels	Fixed sizes (e.g., borders)
pt	Points (1pt = 1/72 inch)	Print stylesheets

4.2 Relative Units

Unit	Description	Use Case
em	Relative to font-size	Responsive typography
%	Percentage of parent	Fluid layouts

4.3 Pro Tips

1. **For responsive design**: Prefer em/rem over px for typography.
2. **Margin/padding shorthand**:

```
margin: 10px 20px; /* Top/Bottom: 10px, Left/Right: 20px */
```

3. **Border shorthand**:

```
border: 1px solid #000; /* width style color */
```

4.4 Practical Example

```
.article {
  font-family: Arial;
  line-height: 1.6;
  width: 80%;
  margin: 0 auto; /* Centers the box */
  padding: 20px;
  border: 1px solid #ddd;}
```

5 Overflow Control

When content exceeds its container's dimensions, use these properties:

Property	Values	Behavior	Example
overflow	visible (default)	Content spills outside	overflow: visible;
	hidden	Hides extra content	overflow: hidden;
	scroll	Always shows scrollbars	overflow: scroll;
	auto	Smart scrollbars (only when needed)	overflow: auto;
word-wrap	break-word	Forces long words to wrap	word-wrap: break-word;

Practical Example:

```
.box {
  width: 250px;
  height: 110px;
  overflow: auto;  /* Adds scrollbars if content overflows */
  word-wrap: break-word; /* Breaks long words */
}
```

- **For modern layouts**: Use overflow: hidden with text-overflow: ellipsis for truncated text:

```
.truncate {
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis; /* Adds "..." */
}
```

6 Color Specification Methods

Method	Syntax	Example	Use Case
Named Colors	Predefined names	color: navy;	Quick prototyping
RGB	rgb(red, green, blue)	rgb(255, 0, 0) (red)	Precise color control
Hex Codes	#RRGGBB	#FF0000 (red)	Web-standard

Color Examples:

```
/* Named color */
h1 { color: teal; }
/* RGB */
button { background: rgb(255, 200, 0); } /* Orange */
/* Hex */
a { color: #FF00FF; } /* Magenta */
```

7 CSS Positioning Guide

7.1 Float Positioning (Legacy Technique)

Given the following HTML document:

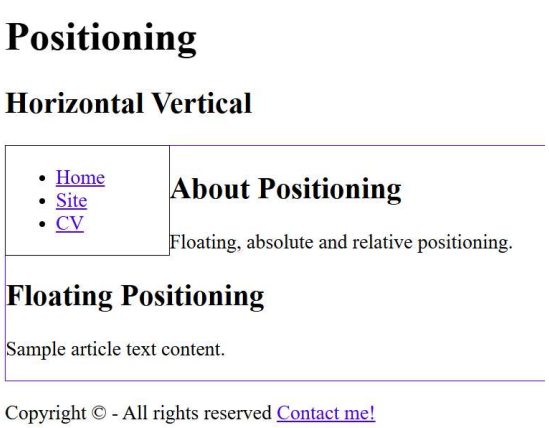
```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8"/>
  <title>Positioning Test</title>
</head>
<body>
  <header>
    <h1>Positioning</h1>
    <h2>Horizontal Vertical</h2>
  </header>

  <main>
    <nav>
      <ul>
        <li><a href="#">Home</a></li>
        <li><a href="#">Site</a></li>
        <li><a href="#">CV</a></li>
      </ul>
    </nav>

    <section>
      <aside>
        <h2>About Positioning</h2>
        <p>Floating, absolute and relative positioning.</p>
      </aside>
      <article>
        <h2>Floating Positioning</h2>
        <p>Sample article text content.</p>
      </article>
    </section>
  </main>
  <footer>
    <p>Copyright © - All rights reserved <a href="#">Contact me!</a></p>
  </footer>
</body>
</html>
```

With these style rules, we will get the following display:

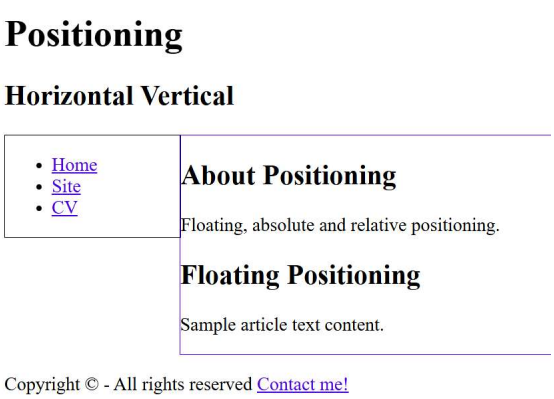
```
nav {
  float: left;
  width: 150px;
  border: 1px solid black;
}
section {
   border: 1px solid blue;
}
```



Result: the navigation menu sticks to the text content without spacing:

Solution: Adding **margin** to fix text overlap

```
nav {
  float: left;
  width: 150px;
  border: 1px solid black;
}
section {
  margin-left: 150px; /* Prevents text collision */
  border: 1px solid blue;
}
```



Problems with Float:

- Requires clearfix hacks (::after { content: ""; clear: both; })
- Disrupts normal document flow

➢ **Avoid for modern layouts**

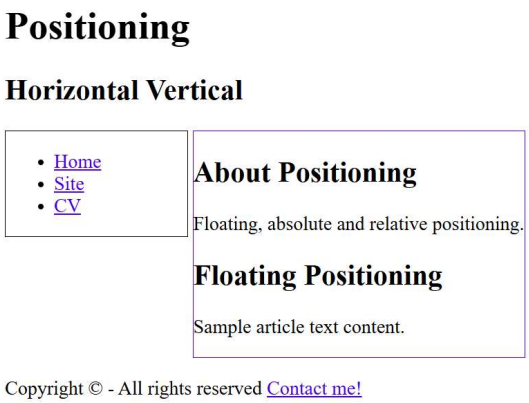
7.2 Display Property

Transform element behavior:

Value	Description	Example
block	Stack vertically (full width)	a { display: block; }
inline	Flow horizontally (no width/height)	span { display: inline; }
inline-block	Flow horizontally + adjustable size	button { display: inline-block; }
none	Hide element	.hidden { display: none; }

Example:

```
nav {
display : inline-block ;
width : 150px ;
border : 1px solid black ;
}
section {
  display : inline-block ;
border : 1px solid blue ;
vertical-align : top ;}
```



7.3 Modern Positioning

Type	Behavior	Example Use
relative	Offsets from normal position	Tooltip positioning
absolute	Positions relative to nearest positioned ancestor	Dropdown menus
fixed	Stays visible during scrolling	Sticky headers
sticky	Hybrid of relative + fixed	Sticky nav bars

Example: Fixed Header

```
header {
  position: fixed;
  top: 0;
  width: 100%;
}
```

7.4 Flexbox (Recommended)

The most recent and powerful CSS technique for positioning elements on a page. It uses a container with child items that can be rearranged purely with CSS (no HTML changes needed).

7.4.1 Core Flexbox Properties

7.4.1.1 Container Properties

Property	Values	Description	Example
display: flex	-	Activates Flexbox	container { display: flex; }
flex-direction	row (default), column, row-reverse, column-reverse	Defines main axis	flex-direction: column;
justify-content	flex-start, center, flex-end, space-between, space-around	Aligns items along main axis	justify-content: center;
align-items	stretch, flex-start, center, flex-end, baseline	Aligns items along cross axis	align-items: center;
flex-wrap	nowrap, wrap, wrap-reverse	Controls line breaks	flex-wrap: wrap;
align-content	flex-start, center, space-between, etc.	Aligns wrapped lines	align-content: space-around;

7.4.1.2 Child Item Properties

Flexbox child properties give precise control over how individual items behave within a flex container.

Property	Description	Example
order	Changes visual order without changing HTML (default: 0)	order: 2;
flex-grow	Controls growth relative to siblings (Controls how items expand to fill space)	flex-grow: 1;
flex-shrink	Controls how items shrink when space is limited	flex-shrink: 0;
flex-basis	Defines default size before growing/shrinking (auto (default) or length (px, %, em)	flex-basis: 200px;
align-self	Overrides container's align-items for specific items	.container { align-items: center; } .special-item { align-self: flex-end; }

7.4.2 Practical Examples

Container Properties:

```
.container {
  display: flex;
  flex-direction: row; /* or column */
  justify-content: center; /* Main axis */
  align-items: center; /* Cross axis */
  flex-wrap: wrap;
}
```

Item Properties:

```
.item {
  flex: 1; /* Grow/shrink */
  order: 2; /* Change visual order */
}
```

Float vs Flexbox:

```
/* OLD (Float) */
nav { float: left; width: 200px; }
main { margin-left: 200px; }

/* NEW (Flexbox) */
body { display: flex; }
nav { width: 200px; }
main { flex: 1; } /* Takes remaining space */
```

7.4.3 Key Takeaways

1. **Replace floats** with Flexbox/Grid
2. **Use inline-block** for simple horizontal layouts
3. **Position carefully**:
    - relative for minor adjustments
    - absolute/fixed for overlays
4. **Flexbox** solves most layout problems

Browser Support:

- Flexbox: All modern browsers
- Avoid floats for new projects

8 Responsive Design

Responsive design ensures your website looks good on all devices.

8.1 Media Queries

Media queries apply styles based on screen size.

Example:

```
/* Default styles */
body {
    font-size: 16px;
}

/* Styles for screens smaller than 600px */
@media (max-width: 600px) {
    body {
        font-size: 14px;
    }
}
```

8.2 Mobile-First Approach

Start with styles for mobile devices and add styles for larger screens.

Example:

```
/* Mobile styles */
.container {
    padding: 10px;
}
/* Tablet and desktop styles */
@media (min-width: 768px) {
    .container {
        padding: 20px;
    }
}
```

9 CSS Priority & Specificity Rules

9.1 Cascade Order (Basic Rule)

When rules conflict, the **last declared style wins** (if specificity is equal).

```
p { color: red; }
p { color: blue; }  /* This one applies */
```

9.2 Specificity Hierarchy

Calculated as (a, b, c, d) where:

- a = Inline styles
- b = IDs
- c = Classes/attributes/pseudo-classes
- d = Elements/pseudo-elements

Selector	Specificity Value	Example
style="..."	(1, 0, 0, 0)	<p style="color:red;">
#id	(0, 1, 0, 0)	#header
.class	(0, 0, 1, 0)	.button
p	(0, 0, 0, 1)	div p

Example:

```
#nav .link { color: blue; }  /* (0,1,1,0) */
a.link { color: red; }       /* (0,0,1,1) - Lower specificity */
```

→ Blue text wins.

9.3 Priority Sources

1. **User !important** (e.g., browser extensions)
2. **Author !important** (your CSS)
3. **Author styles** (your normal CSS)
4. **User styles** (browser defaults)

9.4 !important Rule

Overrides all other declarations:

```
.button { color: red !important; }  /* Wins even over #nav */
```

⚠ **Avoid** unless critical (hard to override).

9.5 Inheritance

Parent styles pass to children unless overridden:

```
<style>
  p { font-size: 20px; }  /* Inherited by <em> */
  em { font-size: 15px; } /* Overrides inheritance */
</style>
<p>Text <em>here</em></p>
```

9.6 Key Takeaways

1. **Order**: Last rule wins (equal specificity).
2. **Specificity**: inline > #id > .class > element.
3. **!important**: use sparingly.
4. **Inheritance**: Children inherit unless styled directly.

Pro Tip: Calculate specificity using this tool: Specificity Calculator.